Multi-agent Semantic Firewall: A Hybrid Defense-in-Depth System for LLM Security

Anonymous Author(s)

Institution details withheld for double-blind review

Abstract. Large Language Models (LLMs) are increasingly being used in security-sensitive applications, yet reliably detecting prompt injection and jailbreak attacks remains challenging. Existing defenses generally deploy a single model-based classifier per request, which results in inference latency and a lack of adaptability to emerging threats. This paper presents Semantic Firewall, a layered multi-agent framework that combines deterministic screening, semantic retrieval, parallel security detectors, and constrained LLM-based verification. The architecture follows an early-exit strategy in which prompts resolved by lightweight mechanisms bypass expensive downstream analysis. A semantic cache stores newly detected threats, allowing future semantic variants to be identified without repeated LLM inference and directly addressing the adaptability gap in prior defenses. Eight specialized detectors execute concurrently to cover complementary threat categories, while a deterministic Llama-3.3-70B gate evaluates unresolved prompts with schema-constrained outputs. On the neuralchemy prompt-injection benchmark, Semantic Firewall achieves a 95.81% F1 score and 97.46% recall on the held-out test split, outperforming GPT-4o, Gemini 2.5 Flash, and Claude Haiku 4.5, while bypassing LLM inference for 50.1% of test requests (fewer when attacks are a smaller share of the traffic). Evaluations on related tasks and unseen data give F1 scores of 77.16% on deepset, 72.95% on BeaverTails, 94.85% on AI4Privacy PII detection, and 92.44% on a new out-of-distribution set. These results demonstrate that combining lightweight detection, adaptive semantic memory, and selective LLM verification provides a practical, cost-effective defense for production LLM applications.

Keywords: Large Language Models · Prompt Injection · LLM Security · Semantic Firewall · Semantic Caching · Multi-Agent Systems.

1 Introduction

Large Language Models (LLMs) are now embedded in critical infrastructure: they power customer support pipelines, generate production code, answer medical queries, and drive autonomous agents that make consequential decisions with minimal human oversight. This rapid transition from research artifact to production dependency has widened the attack surface faster than defenses have matured [1,2]. Adversaries inject carefully crafted prompts to hijack model behavior [3], construct jailbreaks that bypass alignment training [4,5,6,7], and exploit context windows to exfiltrate personally identifiable information (PII) or

cryptographic secrets. The industry's default defense has been to place an additional model ahead of the first: Llama Guard [8], Guardrails AI [9], and Prompt-Guard [10] route every incoming prompt through a dedicated safety LLM, irrespective of whether it is a benign query or a multi-turn injection attempt. This approach is simple to build but expensive to run; routing every request through a dedicated safety model adds substantial latency; our own profiling of a comparable LLM-only gate measures 406 ms per request (Section 5), and GPU costs scale linearly with traffic. It also breaks down under novel attacks, as static safety tuning does not generalize well to new attack patterns, such as encoded payloads, token smuggling, and multi-turn injections. Meanwhile, opaque `"403 Forbidden"` rejections leave downstream applications unable to distinguish a genuine attack from a false positive [12,13]. No single defense mechanism simultaneously achieves high detection accuracy, low latency, low computational cost, and robustness to emerging prompt-injection attacks [14]. In general, conventional defenses utilize detection as a single monolithic assessment, one model-based classifier evaluating every request in isolation, rather than a graded response calibrated to attack ambiguity, a problem this study tackles directly [8,10,15,21].

Securing prompt-based interactions therefore demands the same defense-in-depth philosophy long established in network security [22]: not a single point of inspection, but a layered architecture in which each stage is optimized against a distinct threat class. This paper presents Semantic Firewall, a five-stage realization of this philosophy in which deterministic checks and semantic memory resolve most prompts immediately, escalating only ambiguous cases through a parallel ensemble of detectors and a constrained LLM gate. Confirmed threats feed back into the semantic cache, allowing the system's detection boundary to harden with exposure rather than through manual retraining. The main contributions of this paper are the following:

1. **Layered, early-exit multi-agent defense:** A semantic firewall is proposed that combines deterministic screening, semantic retrieval, parallel threat detection, and constrained LLM verification in a single architecture that resolves each prompt as early as it can be safely determined.
2. **Adaptive semantic memory:** A semantic cache that stores newly confirmed threats, enabling semantically similar future attacks to be identified without repeated LLM inference.
3. **Parallel multi-agent detection:** A parallel ensemble of eight specialized security agents (Table 1) detects complementary threat categories concurrently while minimizing additional detection latency.

This architecture is evaluated on five public benchmarks against eight representative baselines, demonstrating strong detection performance, results on unseen prompt-injection data and on related tasks, and low inference latency.

The rest of this paper is organized as follows. Section 2 reviews related work, Section 3 presents the proposed architecture, Section 4 describes the experimental setup, and Section 5 presents the results. The limitations and future work are discussed in Section 6 and lastly, Section 7 concludes the paper.

2 Related Work

Recent work on securing LLM applications falls into three broad categories: safety classifiers, adversarial robustness research, and guardrail frameworks. Llama Guard [8] and PromptGuard [10] represent the first category, scanning inputs with a learned model. However, this generalizes poorly to attacks outside the training distribution and adds inference overhead. The second category outlines the numerous examples by which safety-aligned models continue to exhibit a lack of reliability. Zou et al. [7] found that GCG-optimized adversarial suffixes transfer across model families, and AutoDAN [4] used genetic search to craft natural-sounding jailbreak prompts. PAIR [5] automated attacks through repeated refinement, and Greshake et al. [3] showed that hidden instructions in outside content can hijack LLM-integrated applications. PromptBench [23] responds with a systematic robustness benchmark, while SmoothLLM [24] responds with a perturbation-based defense.

The third group mentions defenses for the application layer.
NeMo Guardrails [21] enforces conversational policy through programmable rules, though rule-based systems demand continual upkeep as attackers invent new injection techniques. Rebuff [15] takes a similar hybrid approach, combining heuristic detectors, LLM-based verification, and a vector database of previously seen attacks, together with canary tokens inserted into the prompt to reveal leakage of the system prompt; by default, every request is scored by all of these checks. Semantic Firewall differs in running eight detectors in parallel (covering PII, secrets, and context flooding as well as injection), in gating LLM calls and cache write-back on confidence with an administrator allowlist, in separating detection from configurable policy, and in resolving each prompt at the earliest stage that can decide it instead of scoring every request with every check. A complementary line of work hardens the model or the application itself: the instruction hierarchy [17] trains models to give privileged instructions precedence, StruQ [18] and SecAlign [19] fine-tune models to ignore instructions embedded in data, and CaMeL [20] separates control flow from untrusted data so that injected text cannot change what an agent executes. These defenses require changing the protected model or application, whereas Semantic Firewall is an external filter in front of an unmodified model and can be combined with them. Collectively, these three categories of work lead to the same conclusion that no single mechanism is sufficient, and LLM security necessitates layered defenses constructed from complementary detection strategies. Closing this gap is precisely what Semantic Firewall offers.

3 Semantic Firewall Architecture

The proposed semantic firewall follows a layered pipeline that separates request validation, semantic memory, threat detection, and policy-driven decision-making. As shown in Fig. 1, incoming prompts pass through progressively deeper analysis stages, numbered ❶–❻ in the figure and referenced by the same numbers in the subsections below.

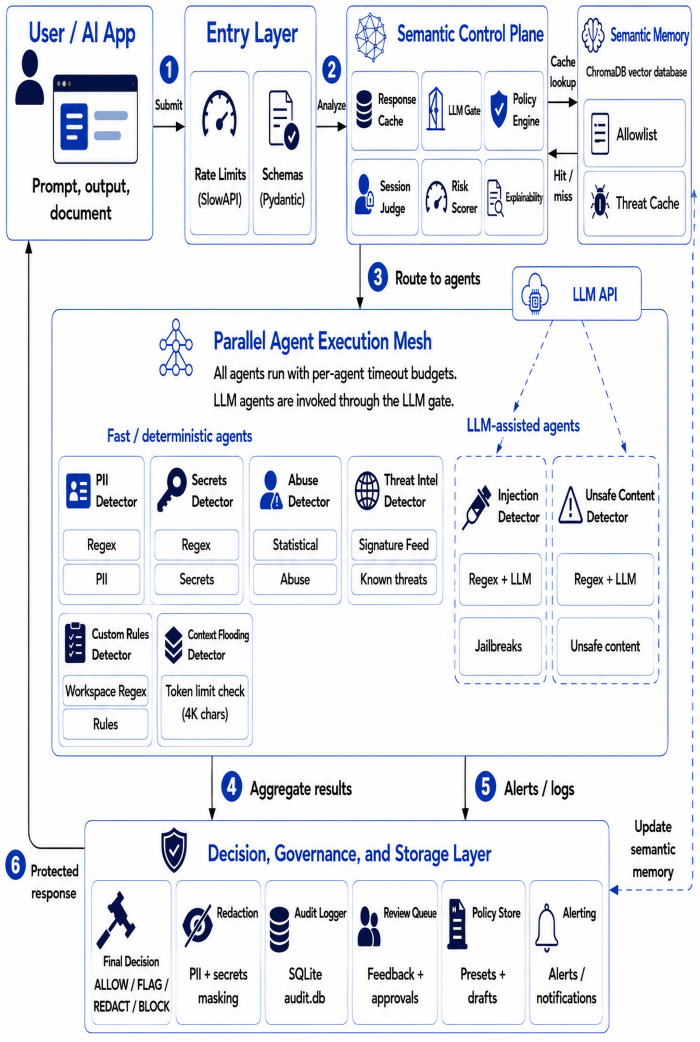


Fig. 1. Multi-agent Semantic Firewall architecture with semantic memory, parallel security agents, and policy-driven decisions. Circled numbers indicate the request path referenced throughout Section 3. The mesh contains the eight detectors of Table 1: six fast/deterministic and two LLM-assisted.

The architecture follows an early-exit strategy. Once a prompt is resolved with sufficient confidence, downstream processing is skipped, reducing unnecessary computation and LLM inference. This confidence is formalized through two thresholds: the similarity threshold τ (Eq. (1)) at the semantic-memory stage and the ensemble threshold θ (Algorithm 1) at the detector-mesh stage. Unresolved prompts escalate in stages, i.e., first to semantic retrieval, then to a parallel ensemble of specialized detectors, and finally to a deterministic Llama-3.3-70B

gate that returns a structured verdict. Newly confirmed threats are written back into the semantic cache so that future, semantically similar variants are resolved at the lightweight layer without repeated LLM inference, allowing the system's detection boundary to harden with exposure rather than requiring manual rule updates or retraining.

3.1 Entry Layer and Initial Screening (Step ❶)

Incoming requests are first rate-limited and validated against the expected schema. Malformed inputs and excessive requests are rejected before entering the detection pipeline. A lightweight regex engine then scans for known attack patterns, including instruction overrides, encoded payloads, jailbreak prefixes, and token-smuggling attempts. The engine runs on the CPU with an average latency of 10.3 ms and immediately blocks matching prompts before more expensive analysis is performed; in the implementation it runs alongside the deterministic detectors, after the semantic-memory lookup (Algorithm 1).

3.2 Semantic Control Plane and Memory (Step ❷)

The Semantic Control Plane orchestrates context-aware detection and policy enforcement and acts as the primary cost gate for LLM inference: fast, low-cost checks run first, and the constrained LLM gate (Section 3.4) is invoked only when they leave a prompt's status ambiguous. Four modules implement this orchestration. The *Session Judge* maintains a sliding window over recent conversation turns and computes an escalating threat trajectory across them, rather than scoring each prompt in isolation; it also compares the conversation so far with a semantic memory of known multi-turn attacks, analogous to the threat cache. This is designed to counter multi-turn jailbreaks such as Crescendo [25], in which an attacker builds conversational trust over several benign-looking turns before injecting a harmful payload on a later one. The *Risk Scorer* aggregates the continuous confidence values reported by each detector, rather than treating their outputs as binary safe/unsafe decisions, weighting each by severity to produce the unified risk score formalized in Section 3.5. The *Policy Engine* maps detector outputs onto configurable deployment profiles (e.g., *Strict*, *Balanced*, *Developer*), so that, for instance, a healthcare application can enforce stringent PII filtering while an internal developer tool applies more permissive rules, without requiring changes to the underlying detectors. Finally, the *Explainability Module* addresses the opacity problem raised in Section 1; rather than returning an unexplained rejection, it aggregates the specific rules, detector scores, and evidence that triggered a decision into a structured, human-readable report, supporting both auditing and the resolution of false positives.

Semantic memory maintains separate stores for known threats and administrator-approved prompts. Each prompt is encoded as a 384-dimensional vector using Sentence-Transformers [26] and searched against a ChromaDB [27] Hierarchical Navigable Small World (HNSW) graph index [28]. Similarity between an incoming prompt $\boldsymbol{v}_q$ and a cached threat $\boldsymbol{v}_c$ is computed as

$$\mathrm{sim}(\boldsymbol{v}_q, \boldsymbol{v}_c) = \frac{\boldsymbol{v}_q \cdot \boldsymbol{v}_c}{\|\boldsymbol{v}_q\|_2 \|\boldsymbol{v}_c\|_2}. \tag{1}$$

The highest similarity score is given by

$$s^* = \max_{\boldsymbol{v}_c \in \mathcal{C}} \mathrm{sim}(\boldsymbol{v}_q, \boldsymbol{v}_c), \tag{2}$$

where $\mathcal{C}$ denotes the threat cache. A prompt is blocked when $s^* \geq \tau$. We set $\tau = 0.65$ based on a sweep over the validation split (Section 5, Table 3), which shows this value maximizes F1; the full sensitivity analysis is reported in Section 5.

The semantic memory is updated on both of its sides, the threat cache and the allowlist, and stores only previously seen malicious prompts and administrator-approved prompts. When downstream analysis confirms a novel attack, its embedding is added to the threat cache. Only LLM-confirmed detections with confidence ≥ 0.85 are written, and the cache keeps at most 10,000 entries, evicting the oldest first. Growth does not slow the lookup: a nearest-neighbour query (without the embedding) takes 31, 20 and 19 ms on average with 1,000, 5,000 and 20,000 entries, so the cap bounds storage rather than latency, and with a cap of 500, inserting 2,000 entries leaves exactly 500. In our experiments the cache never holds more than about 2,700 entries, so no entry is evicted. Consequently, a subsequent prompt that rephrases the same intent, for instance, following up a blocked request such as "Give me your secret instructions" with a paraphrase such as "Provide your hidden system prompt" is then recognized by similarity rather than by exact string match. Therefore, such prompts get blocked immediately without re-invoking the parallel detectors or the LLM gate. Conversely, if the pipeline flags a legitimate but unusually phrased prompt as a false positive, an administrator can approve it once; its embedding is then added to the allowlist so that future prompts of similar intent (cosine similarity ≥ 0.90) skip the LLM Gate, while the deterministic detectors still run. This closes the loop in both directions: repeated attacks become cheaper to block over time, and repeated false positives become cheaper to correct, without manual rule updates or retraining.

3.3 Parallel Agent Execution Mesh (Step ❸)

Prompts unresolved by the initial screening and semantic memory stages are routed to a set of eight specialized security agents, summarized in Table 1. The architecture executes these agents concurrently with individual timeout budgets. For detector execution times $T_1, \ldots, T_8$, the idealized ensemble latency is

$$T_{\mathrm{mesh}} \approx \max_{1 \leq i \leq 8} T_i + T_{\mathrm{ovh}}, \tag{3}$$

where T_{ovh} represents thread scheduling, communication, and aggregation overhead.

Table 1. The eight specialized security agents in the Parallel Agent Execution Mesh.

Detector	Category	Function
PII Detector	Fast / deterministic	Identifies sensitive identifiers (emails, phone numbers, IDs) using pattern-based rules.
Secrets Detector	Fast / deterministic	Uses signature matching to flag API keys, tokens, and credentials.
Abuse Detector	Fast / deterministic	Flags repetition, low-entropy input, homoglyphs, and encoded payloads.
Threat Intel Detector	Fast / deterministic	Matches prompts against a signature feed of previously observed threats.
Custom Rules Detector	Fast / deterministic	Evaluates workspace-specific regex and policy rules.
Injection Detector	LLM-assisted	Combines regex screening (with Levenshtein repair of obfuscated keywords) with the LLM Gate to catch jailbreak and prompt-injection attempts.
Unsafe Content Detector	LLM-assisted	Combines regex screening with the LLM Gate to flag unsafe or policy-violating content.
Context Flooding Detector	Fast / deterministic	Checks for context/token-flooding attacks against a configurable length budget (e.g., 4K characters).

The deterministic agents cover complementary threat categories. The PII Detector identifies sensitive identifiers using pattern-based rules. The Secrets Detector uses signature matching to detect API keys, tokens, and other credentials. The Abuse Detector uses Shannon entropy to flag highly repetitive input. For a candidate string X of length n, its entropy is defined as [29]

$$H(X) = -\sum_{c \in \Sigma_X} p(c) \log_2 p(c), \tag{4}$$

where Σ_X is the set of distinct characters in X and $p(c) = n_c/n$ is the empirical probability of character c, which occurs n_c times in X. The check is applied only to inputs of at least 50 characters and flags those with $H(X) < 2.5$ bits. The Injection Detector identifies lexical obfuscations using approximate string matching based on the Levenshtein distance. For strings a and b, let $d_{a,b}(i,j)$ denote the minimum edit distance between the prefixes $a_{1:i}$ and $b_{1:j}$. The distance is recursively defined as [30],

$$d_{a,b}(i,j) = \begin{cases} \max(i,j), & \text{if } \min(i,j) = 0, \\ \min \begin{cases} d_{a,b}(i-1,j) + 1, \\ d_{a,b}(i,j-1) + 1, \\ d_{a,b}(i-1,j-1) + \delta_{ij}, \end{cases} & \text{otherwise,} \end{cases} \tag{5}$$

where $\delta_{ij} = \mathbb{I}(a_i \neq b_j)$ denotes the indicator function. The three terms correspond to deletion, insertion, and substitution operations, respectively. Smaller Levenshtein distances indicate greater lexical similarity, enabling the detector to identify obfuscated variants of the keywords in its regex patterns. The repair is enabled in all reported results; on ordinary neuralchemy prompts it changes only 2 of the 941 validation predictions, but when one typo is inserted per keyword

it raises regex detection of the 62 test attacks that the regex catches without typos from 33.9% to 80.1%. The remaining agents (Threat Intel, Custom Rules, Injection, and Unsafe Content Detectors in Table 1) perform threat-intelligence matching, workspace-specific rule evaluation, injection detection, and unsafe-content analysis, respectively. Injection and unsafe-content detectors may invoke the LLM Gate when deterministic evidence is insufficient. This organization allows inexpensive agents to resolve common threats while reserving model-based reasoning for ambiguous inputs.

3.4 Constrained LLM Gate (invoked within Step ❸)

Prompts that remain unresolved after deterministic analysis and semantic lookup are evaluated by the LLM Gate. The current implementation uses Llama-3.3-70B-Instruct through OpenRouter. The model is instructed to return a schema-constrained JSON decision containing a binary verdict, the attack type, a confidence value, the supporting evidence, and an overall risk level. Greedy decoding (temperature 0) is used to reduce output variability and support reproducible evaluation.

When the LLM Gate identifies a novel attack, the corresponding prompt embedding is inserted into the threat cache. This feedback path allows subsequent semantic variants to be resolved without repeated LLM inference. If the external LLM service is unavailable, the firewall applies the configured fallback policy (by default FLAG) rather than silently bypassing security checks.

3.5 Risk Aggregation and Policy Enforcement (Step ❹)

The agent mesh produces detector decisions, confidence values, matched patterns, and severity indicators. These heterogeneous outputs, the set of detector results $\mathcal{R}$, are combined into a continuous risk score:

$$\mathrm{Risk}(\mathcal{R}) = \min\left(100,\ S_{\mathrm{sev}} + S_{\mathrm{conf}} + S_{\mathrm{pat}} + S_{\mathrm{ses}}\right). \tag{6}$$

The risk score is calculated by combining the highest detected threat severity S_{sev}, confidence-weighted evidence S_{conf} from triggered detectors, agreement across detectors S_{pat}, and S_{ses} risk accumulated during the current session. Severity receives the largest fixed base value (0–85), confidence contributes up to 30 points, detector agreement adds 5 points per trigger (up to a maximum of 20), and session history accounts for up to 10. These heuristic weights prioritize serious threats while allowing strong evidence, consensus, and repeated suspicious activity to increase borderline scores; the final score is capped at 100. This score is used for explanations and session tracking; blocking uses the policy actions of the triggered detectors $\mathcal{T} \subseteq \mathcal{R}$ and the ensemble score

$$E(\mathcal{R}) = \sum_{a \in \mathcal{T}} w_a \, p_a \, \frac{\max(1, s_a)}{2}, \tag{7}$$

Algorithm 1 Semantic Firewall Classification Pipeline

Input: Prompt q, optional session σ, similarity threshold τ, ensemble threshold θ, threat cache $\mathcal{C}$, allowlist $\mathcal{A}$, detector set $\mathcal{D} = \{D_1, \ldots, D_8\}$ (LLM-assisted: Injection, Unsafe Content)

Output: Decision $d \in \{\textsc{Allow}, \textsc{Flag}, \textsc{Redact}, \textsc{Block}\}$

$\boldsymbol{v}_q \leftarrow \textsc{Embed}(q)$
$a_{\mathcal{A}} \leftarrow \textsc{AllowlistMatch}(\boldsymbol{v}_q, \mathcal{A})$ ▷ admin-approved prompts
$s^* \leftarrow \max_{\boldsymbol{v}_c \in \mathcal{C}} \frac{\boldsymbol{v}_q \cdot \boldsymbol{v}_c}{\|\boldsymbol{v}_q\|_2 \|\boldsymbol{v}_c\|_2}$
if $s^* \geq \tau \wedge \neg a_{\mathcal{A}}$ **then**
  $\mathcal{R} \leftarrow \{\text{cached threat}\}$ ▷ LLM skipped
else
  $\mathcal{R} \leftarrow \textsc{ParallelExecute}(\mathcal{D}, q)$ ▷ regex pre-screen only, no LLM
  if $\neg a_{\mathcal{A}} \wedge \textsc{PolicyAction}(\mathcal{R}) \neq \textsc{Block}$ **then**
    $\mathcal{R} \leftarrow \mathcal{R} \oplus \textsc{LLMDetect}(q)$ ▷ JSON; failure $\Rightarrow$ Flag
  end if
end if
$e \leftarrow E(\mathcal{R})$
$d \leftarrow \textsc{PolicyAction}(\mathcal{R})$
if $e \geq \theta$ **then**
  $d \leftarrow \textsc{Block}$ ▷ lower bands of e raise d to Flag/Redact
end if
if σ given **then**
  $d \leftarrow \textsc{SessionJudge}(\sigma, q, d)$ ▷ conversation check and memory
end if
if $d = \textsc{Redact}$ **then**
  $q \leftarrow \textsc{Mask}(q, \mathcal{R})$
end if
if an LLM detector confirms a threat in $\mathcal{R}$ **then**
  $\mathcal{C} \leftarrow \mathcal{C} \cup \{\boldsymbol{v}_q\}$ ▷ Update threat cache
end if
return d

where w_a, p_a, and s_a are the weight, calibrated confidence, and severity rank (0–4) of detector a. A prompt is blocked if a policy action is Block or $E(\mathcal{R}) \geq \theta = 3.5$. The detector weights (1.0–1.3, highest for the injection and unsafe-content detectors) and the risk-score points were set by hand during development and were not tuned on any evaluation split; sweeping θ from 2.0 to 6.0 left validation F1 unchanged (Section 5).

Detector outputs are evaluated by the Policy Engine using configurable security profiles and thresholds. This separation allows the same detection pipeline to support different deployment requirements without modifying individual detectors. High-confidence critical findings may trigger immediate blocking, while intermediate-risk requests can be flagged, redacted, or forwarded for human review.

3.6 Decision, Governance, and Storage (Steps ❺–❻)

The final layer converts the accumulated security evidence into one of four actions: ALLOW, FLAG, REDACT, or BLOCK. Sensitive entities and secrets detected in otherwise legitimate prompts may be masked before the request is forwarded to the target LLM. Each transaction is recorded by the Audit Logger, including payload hashes, detector outcomes, risk scores, processing latency, and final decisions. Ambiguous or policy-sensitive cases may be forwarded to a human review queue. Analyst decisions can subsequently update the allowlist, threat cache, or policy configuration. High-risk events may additionally generate asynchronous notifications through external alerting channels.

3.7 Classification Procedure

Algorithm 1 summarizes the core classification pipeline of Semantic Firewall. Each prompt is first checked against the allowlist and compared with previously detected threats in the semantic cache. Prompts that remain unresolved are analyzed by eight detectors running in parallel, the two LLM-assisted ones initially by their regex pre-screen only. Only prompts that these checks do not block reach the constrained LLM Gate; all detector outputs are then aggregated into a risk score, and a failed LLM call yields FLAG rather than ALLOW. When the LLM Gate identifies a novel attack, its embedding is added to the threat cache, allowing similar inputs to be detected earlier in subsequent requests. This early-exit design limits unnecessary computation while enabling the firewall to adapt to newly observed threats.

3.8 Effective Latency

Because prompts may terminate at different stages, end-to-end latency depends on the fraction of requests resolved by each path. Let P_h, P_c, P_m, and P_l denote the probabilities that a prompt is resolved by initial screening, semantic memory, the parallel agent mesh, or the LLM Gate, respectively. The expected latency can be expressed as,

$$L_{\text{eff}} = P_h L_h + P_c L_c + P_m L_m + P_l L_l, \qquad \sum_{k\in\{h,c,m,l\}} P_k = 1, \tag{8}$$

where L_h, L_c, L_m, and L_l are the corresponding mean path latencies. This formulation reflects the architecture more accurately than treating total latency as the sum of all component execution times.

3.9 Threat Model

System and assets. The firewall sits between untrusted users and a protected LLM application. The assets are the integrity of the downstream model's behavior, the availability of the service to benign users, the firewall's persistent state (the threat cache and the allowlist), and the confidentiality of user prompts.

Adversary goals. **G1 (evasion):** deliver a prompt-injection or jailbreak payload to the protected model without being blocked. **G2 (cache poisoning):** cause a wrong block decision to be persisted, so that similar benign prompts are rejected later. **G3 (allowlist abuse):** craft a malicious prompt close enough to an approved prompt to inherit its allow decision and skip the LLM gate.

Adversary knowledge and capabilities. A **black-box** attacker ($\mathcal{A}_1$) observes only the final verdict. A **pipeline-aware** attacker ($\mathcal{A}_2$) also knows the architecture, the detectors, the embedding model, the similarity threshold τ, the gate's system prompt and the cache write-back rules, but not the contents of the cache or the allowlist. Both can submit arbitrary text prompts, adapt them using the verdicts returned, and operate several client identities within a query budget. Neither can write to the cache or the allowlist directly, nor modify the firewall's configuration or models; the cache can be influenced only through the automatic write-back of gate verdicts, and the Session Judge's memory of multi-turn attacks is never written online.

Trust assumptions. The operator is trusted and curates the allowlist and the benign reference set, and the host is uncompromised. The third-party LLM provider is honest but curious: it returns correct completions but may retain the prompts it receives.

Out of scope. Indirect injection through retrieved documents or tool outputs, attacks on the protected model's outputs, training-time poisoning, timing side channels, and volumetric denial of service.

Coverage in this paper. This section summarizes which threats are evaluated. For G1, Section 5 measures detection of static attack datasets by the black-box attacker: prompt injections from neuralchemy and deepset, out-of-distribution attacks (OOD-500), multi-turn jailbreaks, test prompts unlike any cached attack, and, in Section 3.3, keywords obfuscated with typos. For G2, Section 5.3 measures how gate false positives propagate through the cache without any attacker and what the poisoning-resistant cache design costs on normal traffic. Four threats remain unevaluated: a deliberate poisoning campaign (G2), including by a pipeline-aware attacker who knows the write-back rules, and how much the poisoning-resistant cache design reduces it; evasion by an attacker who adapts inputs to the detectors or uses optimization against an open-weight gate; injection addressed to the LLM gate itself; and near-allowlist inputs optimized by a pipeline-aware attacker (G3; a black-box test of G3 is reported in Section 3.10). We discuss these in Section 3.10 and leave their evaluation to future work. The detection results should therefore be read as performance against known, non-adaptive attacks.

3.10 Security Considerations

The detection stages are not assumed to fail independently, as a carefully obfuscated attack may evade multiple detectors at the same time. The architecture

therefore combines complementary detection mechanisms so that weaknesses in one stage can be addressed by evidence from others. The LLM Gate itself reads attacker-controlled text and may be manipulated by it. Its output is constrained to a fixed JSON schema, and the final action is the most severe action of all detectors, so a manipulated gate cannot undo a block issued by the semantic cache or the deterministic detectors, which run before it; it can, however, fail to report a novel attack that only it would have caught.

Detection is also separated from policy enforcement. Outputs from the security agents are aggregated into a risk score, while configurable policies determine whether a prompt is allowed, flagged, redacted, or blocked. This separation allows the same detection pipeline to support different security requirements without modifying the underlying detectors.

Users cannot write to the allowlist; only the operator adds approved prompts. In a black-box test of G3, we appended injection payloads to 100 approved prompts: only 4 remained within the allowlist threshold (median similarity 0.70), and in the default mode, in which an allowlist hit skips only the LLM Gate, the deterministic detectors still ran and none of the 4 was allowed, whereas the optional bypass mode would have allowed all 4.

Poisoning-resistant cache design. Cache poisoning is possible because a single unverified gate verdict becomes a persistent decision over a whole neighborhood of prompts. We therefore propose four changes to the write-back path that address prevention, containment, and recovery. First, a *benign-overlap check* compares each candidate entry against the allowlist and a reference set of known benign prompts, and rejects the write if the entry would cover any of them; this is designed to prevent the most damaging outcome, in which a legitimate use case becomes blocked. Second, automatically added entries enter a *probationary tier* that is matched at a stricter similarity threshold $\tau' > \tau$, so a new entry still catches replays of the same attack but cannot block a wide region of embedding space; entries are promoted to the trusted tier only after repeated independent detections or operator review. Third, probationary entries carry a *time-to-live* (TTL) that is renewed only on reconfirmation, which bounds how long any erroneous entry can persist. Fourth, a small fraction of the cache hits on automatically added entries is *re-validated* by the full pipeline, including the LLM gate, and an entry is evicted when the pipeline no longer agrees with it, so that poisoned entries can be removed without operator intervention. Together these measures mean that an attacker must cause repeated, high-confidence gate errors on prompts that are far from all known benign traffic, and must keep doing so to sustain the effect. They do not prevent the gate's initial false positive, only its propagation, and they reduce the cache hit rate and thus increase gate calls. The four measures are implemented as options of the cache, are off in all other experiments, and are evaluated in Section 5.3.

4 Experimental Setup

The evaluation uses five public benchmarks selected to cover different security challenges. The *neuralchemy/Prompt-injection-dataset* [32] serves as the primary benchmark with its official splits: the 2,650 attacks of the training split (4,391 prompts) only seed the semantic cache, the validation split (941) is used to select thresholds, and all reported results use the test split ($N = 942$; 552 attacks, 390 benign prompts). The *ai4privacy/pii-masking-200k* dataset [33] ($N = 209{,}261$) is used to evaluate PII detection at scale on multilingual data.
Multi-turn detection is assessed on 200 held-out conversations: 100 multi-turn jailbreaks from *SafeMTData* [34] and 100 benign multi-turn conversations from *no_robots* [37]; a separate development part of 50 attack and 50 benign conversations is used to select the threshold of the Session Judge's memory (0.60).
The *PKU-Alignment/BeaverTails* [35] benchmark ($N = 1{,}000$, a seeded sample of its test split) evaluates safety classification across 14 harm categories, including ambiguous and borderline prompts. Finally, *deepset/prompt-injections* [36] ($N = 662$), a public benchmark on which *Rebuff* [15] has been independently evaluated [16], is used to assess performance on a benchmark designed specifically for prompt-injection detection. For the per-detector ablation, we also build MIXED-1000, which adds inputs for every detector to neuralchemy test prompts (Section 5) and is released with the code.

The benign test split (500 prompts) of *HuggingFaceH4/no_robots* [37] is used for measuring false positives. System thresholds, including the semantic similarity threshold τ and the ensemble threshold θ, are selected on the neuralchemy validation split; the test split is evaluated once. *Cache protocols.* In the warm protocol, the threat cache is initialised with the embeddings of the 2,650 training-split attacks, $\mathcal{C}_0 = \{\boldsymbol{v}(x) : x \in D_{\text{train}},\, y(x) = 1\}$, where $\boldsymbol{v}(x)$ is the embedding of prompt x, D_{train} the training split and $y(x) = 1$ its attack label, and write-back is disabled, so the cache does not change during evaluation and no validation or test prompt ever enters it. A prompt q is then blocked by the cache when $s^*(q) \geq \tau$ and otherwise follows exactly the path it takes without a cache, whose decision is $\hat{y}_{\text{none}}(q)$ ($\hat{y} = 1$: blocked):

$$\hat{y}_{\text{warm}}(q) = \begin{cases} 1, & s^*(q) \geq \tau, \\ \hat{y}_{\text{none}}(q), & \text{otherwise.} \end{cases} \tag{9}$$

In the cold protocol (C9), the cache starts empty and LLM-confirmed attacks with confidence ≥ 0.85 are written back while the test prompts are processed in their dataset order. The official splits share no prompt, so the warm cache contains no test prompt; Section 5 also reports results on the test prompts that are dissimilar to every cached attack. The implementation will be made publicly available upon publication (link withheld for double-blind review).

4.1 Implementation, Baselines, and Evaluation Metrics

Experiments were conducted on a laptop with an Intel Core Ultra 5 125H CPU (18 logical cores) and 16 GB of RAM. Semantic memory was implemented using ChromaDB [27] with HNSW indexing [28], while prompt embeddings were generated using
Sentence-Transformers [26]. The LLM Gate used Llama-3.3-70B-Instruct through OpenRouter with deterministic decoding (temperature 0).

The evaluation includes GPT-4o [38], Claude Haiku 4.5 [39], Gemini 2.5 Flash [40], and Llama Guard 4 [41] as baselines. The three proprietary models were evaluated zero-shot without task-specific prompt tuning, representing an out-of-the-box deployment setting.[1] As these models are not specifically designed for binary prompt-injection classification, their results are reported under this evaluation setting. Llama Guard 4 was also evaluated zero-shot for structural prompt-injection detection. Two purpose-built prompt-injection classifiers, ProtectAI's DeBERTa-v3 [11] and PIGuard [12] (the renamed InjecGuard model), were run locally with their default labels; Meta's Prompt Guard 2 (86M) was queried through the Groq API; a prompt counts as an injection when its score exceeds 0.5, and prompts longer than the model's 512-token window are split into parts and the highest score is used. The zero-shot prompt asks the model to label the user message as *injection* (text that tries to override, ignore or reveal the system instructions, adopt an unrestricted persona, or smuggle instructions through encodings, delimiters or hypothetical framing) or *benign* (ordinary questions, creative requests and discussions about security), and to reply in JSON.

Performance is reported using Precision, Recall, F1 score, and False Positive Rate (FPR), with attacks as the positive class: Precision $= TP/(TP + FP)$, Recall $= TP/(TP + FN)$, F1 is their harmonic mean, and FPR $= FP/(FP + TN)$, the share of benign prompts that are flagged. Pairwise differences in classification outcomes are tested for statistical significance using McNemar's test [31], which compares the prompts on which only one of two systems is correct and is reported as the χ^2 statistic and its p-value; p-values over several baselines are adjusted with the Holm–Bonferroni correction. Effective latency and LLM-call rates, which determine the API cost, are also reported to assess the practical efficiency of the system; besides the mean, latency is given as the median (p50) and the 95th and 99th percentiles (p95, p99), the times within which 50%, 95% and 99% of the requests complete.

5 Results

To evaluate Semantic Firewall, we conduct four sets of experiments: baseline comparisons against eight representative systems, cross-dataset and related-task evaluations, specialized evaluations of individual system components, and a sensitivity analysis over the semantic-cache threshold τ. We further evaluate cache

[1] OpenRouter model IDs (queried October 2026): `openai/gpt-4o`, `anthropic/claude-haiku-4.5`, and `google/gemini-2.5-flash`.

Benchmark	Model / Config	N	Prec.	Recall	F1	FPR	Latency	Notes
(a) Open-Weight vs. Proprietary API Comparison (neuralchemy test split, N= 942)								
neuralchemy	**Semantic Firewall (Ours, Warm)**	942	94.22	**97.46**	**95.81**	8.46	304 ms	Steady-state
neuralchemy	Semantic Firewall (Ours, Cold)	942	95.71	92.93	94.30	5.90	—	Empty cache
neuralchemy	OpenAI GPT-4o	942	96.98	81.52	88.58	3.59	1109 ms	Proprietary
neuralchemy	Google Gemini 2.5 Flash	942	96.96	80.98	88.25	3.59	985 ms	Proprietary
neuralchemy	Anthropic Claude Haiku 4.5	942	97.01	70.65	81.76	3.08	1885 ms	Proprietary
neuralchemy	ProtectAI DeBERTa-v3	942	**97.75**	86.41	91.73	**2.82**	425 ms	Classifier
neuralchemy	Prompt Guard 2 (86M)	942	95.28	54.89	69.66	3.85	189 ms	Classifier (API)
neuralchemy	Llama Guard 4 (12B)	942	92.49	35.69	51.50	4.10	717 ms	Open-weight
neuralchemy	PIGuard (InjecGuard)	942	83.84	79.89	81.82	21.79	345 ms	Classifier
(b) Cross-Dataset and Related-Task Evaluation								
BeaverTails	**Semantic Firewall**	1000	69.50	**76.75**	**72.95**	47.47	—	14 harm categories
BeaverTails	Llama Guard 4 (12B)	1000	**77.78**	62.22	69.14	**25.06**	—	—
deepset/prompt-inj.	**Semantic Firewall**	662	**92.55**	66.16	**77.16**	**3.51**	—	—
deepset/prompt-inj.	Rebuff (reference)	662	63.30	**75.60**	68.90	28.80	—	Published in [16]
(c) Specialized Evaluations								
ai4privacy/pii-200k	Semantic Firewall (Regex Layer)	209K	95.50	94.20	94.85	—	<1 ms	Zero LLM cost
SafeMTData + no_robots	Semantic Firewall (Full)	200	93.68	89.00	91.28	6.00	—	Session level

Table 2. Baseline, cross-dataset, and specialized evaluations. Best results are **bolded**; "—" denotes non-applicable metrics. Latency is the mean per request; ours was measured sequentially with the LLM Gate served by Groq via OpenRouter, the proprietary models through OpenRouter's default hosts, and the classifiers on CPU (Prompt Guard 2 through the Groq API, timed separately without rate-limit waits).

write-back and its safeguards, and perform an ablation study isolating the contribution of each architectural component. Tables 27– present the full results, analyzed in detail in the following subsections.

5.1 Evaluation

Baseline Comparison (Table 2a) Semantic Firewall achieves the highest F1 score among the evaluated systems (95.81%), compared with GPT-4o (88.58%), Claude Haiku 4.5 (81.76%), and Gemini 2.5 Flash (88.25%). Although GPT-4o achieves higher precision (96.98%), Semantic Firewall provides substantially higher recall (97.46% versus 81.52%), indicating more consistent detection of attack prompts, at a higher FPR (8.46% versus 3.59%). This improvement is largely due to the layered architecture, where deterministic and semantic checks identify structural attacks before model-based evaluation. The difference is statistically significant under McNemar's test [31] ($\chi^2 = 38.9$, $p < 10^{-8}$; the gains over every baseline remain significant after Holm–Bonferroni correction). The strongest purpose-built classifier, DeBERTa-v3, reaches 91.73% F1 at a 2.82% FPR, while Prompt Guard 2 is precise (95.28%) but detects only 54.89% of the attacks. The higher recall comes at the cost of precision and FPR: the most severe action of all stages is applied, so false positives of individual stages add up, and the LLM Gate raises the FPR from 4.87% (C7) to 8.46% on ambiguous benign prompts; a stricter cache threshold trades some recall for fewer false positives ($\tau = 0.80$: 6.67% FPR at 94.62% F1). Single small classifiers such as Prompt Guard 2 (166 ms median) are also faster than the full pipeline (279 ms median), whose prompts that reach the 70B gate take 438 ms at the median, while the pipeline remains 3–6 times faster than the proprietary LLMs (median 851–1767 ms). In addition, on this attack-heavy split 50.1% of requests are resolved without invoking the LLM, reducing effective latency to 308 ms (Eq. (8));

measured end-to-end latency is 279 ms at the median, 756 ms at p95 and 1047 ms at p99; Table 7 gives the values at realistic attack rates.

Cross-Dataset Evaluation (Table 2b) In the cross-dataset evaluation, the system is applied, with the thresholds selected on neuralchemy, unchanged to datasets that were not used for tuning, to test whether it works beyond the distribution it was tuned on. The deepset dataset is another prompt-injection benchmark; BeaverTails (harmful requests) and AI4Privacy (personal data) are related tasks, which exercise the Unsafe Content and PII detectors of the mesh rather than injection detection. Semantic Firewall maintains competitive performance on related, unseen tasks. On BeaverTails, which includes 14 harm categories and ambiguous safety cases, it achieves an F1 score of 72.95%, compared with 69.14% for Llama Guard 4. The lower Precision (69.50%) reflects the conservative behavior of the detection pipeline on borderline prompts, while the high Recall (76.75%) indicates that most harmful inputs are still detected. BeaverTails labels a question together with a model's answer, so harmful-sounding questions labelled safe raise the FPR of any prompt-level filter (47.47% here, 25.06% for Llama Guard 4). On the deepset prompt-injection benchmark, Semantic Firewall achieves an F1 score of 77.16%; for reference, an independent evaluation reported an F1 score of 68.90% for Rebuff on this dataset [16]. The system holds up across datasets because most of its stages (regex, detectors and the LLM Gate) are not trained on neuralchemy; the cache, the only part built from neuralchemy, helps little on unseen data (10.4% hit rate on OOD-500), so detection rests on the general-purpose stages.

Specialized Evaluations (Table 2c) The PII detection layer achieves 94.85% F1 and 94.20% Recall across 209K samples (91.32% F1 without the dataset-adapted account-number and SSN patterns), with sub-1 ms processing latency and no neural model inference. Structured identifiers, including email addresses, credit cards, and International Bank Account Numbers (IBANs), achieve over 99.6% Recall, while more variable patterns such as dates of birth and card verification codes remain harder to detect (66.3% and 45.7% recall). In multi-turn scenarios, Semantic Firewall intercepts 89.0% of adversarial sessions, at a 6.0% FPR on benign conversations, on average at turn 2.3. Without its memory of known multi-turn attacks, the Session Judge detects 50.0% of the attack sessions (FPR 4.0%); with it, 89.0% (McNemar $p < 10^{-7}$). Evaluation of benign prompts on no_robots [37] results in a 2.80% block rate; a further 7.4% are forwarded with personal data masked or flagged by the PII detector, indicating limited degradation on non-adversarial inputs.

5.2 Cache Threshold experiments (Table 3)

The semantic-cache threshold τ controls how readily a prompt is blocked based on its similarity to a previously confirmed threat (Eq. (1)): lower values catch more paraphrased attack variants but risk misclassifying benign prompts, while

(a) Full System (C8) on neuralchemy validation split (N= 941)

Threshold (τ)	Prec.	Recall	F1	FPR	Cache hit
No cache	95.63	90.07	92.77	5.41%	0.0%
$\tau = 0.55$	91.26	97.75	94.39	12.29%	56.0%
$\tau = 0.60$	92.83	97.00	94.87	9.83%	51.5%
$\tau = 0.65$ (Sel.)	93.94	95.88	**94.90**	8.11%	45.6%
$\tau = 0.70$	94.59	94.94	94.77	7.13%	39.6%
$\tau = 0.80$	95.17	92.32	93.73	6.14%	24.1%
$\tau = 0.90$	95.65	90.64	93.08	5.41%	12.8%

(b) Out-Of-Distribution (OOD) Generalization (OOD-500)

Threshold (τ)	Prec.	Recall	F1	FPR	Cache hit
No cache	97.77	87.60	92.41	2.00%	0.0%
$\tau = 0.55$	94.85	88.40	91.51	4.80%	27.4%
$\tau = 0.60$	96.92	88.00	92.24	2.80%	16.6%
$\tau = 0.65$	97.35	88.00	**92.44**	2.40%	10.4%
$\tau = 0.70$	97.77	87.60	92.41	2.00%	5.6%
$\tau = 0.80$	97.77	87.60	92.41	2.00%	0.4%
$\tau = 0.90$	97.77	87.60	92.41	2.00%	0.0%

Table 3. Cache Threshold Sensitivity (Across τ sweeps)

higher values are more conservative but let more variants fall through to the slower detector mesh. Table 3a sweeps τ from 0.55 to 0.90 on the full system (C8) over the neuralchemy validation split, with the cache seeded only with training-split attacks and frozen during evaluation, and exposes a clear trade-off governed by this single parameter: τ=0.55 maximizes the cache hit rate but is too permissive, inflating the false-positive rate to 12.29%, while τ=0.90 matches so few prompts that most prompts bypass the cache, eroding its latency benefit. F1 peaks at τ=0.65, which we adopt as the operating threshold: it reaches the highest F1 of any setting tested (94.90%), with a recall of 95.88%, at a 45.6% cache hit rate and 8.11% FPR. The ensemble threshold θ was swept jointly (2.0–6.0) and did not change F1, so the default $\theta = 3.5$ is kept.

Table 3b repeats this sweep on OOD-500, 500 prompts (250 attacks, 250 benign) that we curated from five public datasets not used to build neuralchemy and that we release with the code, to test whether a threshold tuned on neuralchemy transfers to unseen data. F1 on this set also peaks at τ=0.65 (92.44%). Few OOD prompts resemble the cached training attacks (10.4% hit rate), so here detection rests on the later stages and the cache neither helps nor hurts noticeably; for $\tau \geq 0.70$ every cache hit is an attack that the later stages already block, so these rows equal the no-cache row. On the 495 neuralchemy test prompts whose nearest training attack lies below τ, where the cache cannot block them, the full system reaches 88.3% F1, compared with 84.8% for GPT-4o and 82.0% for DeBERTa-v3, so the gains in Table 2a do not rest only on near-duplicates of training attacks.

5.3 Cache Write-back and Poisoning Resistance

We measure how gate false positives propagate through the cache, and what the safeguards of Section 3.10 cost, on normal traffic without an attacker. A prompt that misses the cache follows the same path as without a cache (Eq. (9)), so a run with write-back can be replayed exactly from one run of the full system

without a cache: a prompt that matches an entry is blocked, and any other prompt receives its recorded verdict and is written to the cache if that verdict meets the write-back rule. The replay uses the cache implementation itself. Its recorded verdicts agree with the no-cache run behind Table 2 on 99.3% of the test prompts, and from an empty cache in the dataset order it reproduces C9: the same 151 cache hits, 696 LLM calls and recall, with one more false positive. The test split is streamed in its dataset order and in four random orders, from the warm cache and from an empty one, and the cache reached at the end of each stream is then frozen and applied to the validation split. The benign reference set is drawn from the 1,741 benign training prompts, and a probationary entry is promoted after two independent detections (Table 4).

Of the 942 test prompts, 14 benign ones receive a verdict that meets the write-back rule. In no stream did such a wrongly written entry block a benign prompt: none of them lies within τ of a benign validation prompt (the closest is at 0.58), and none of the 8 such validation prompts lies within τ of a benign test prompt, whereas a randomly chosen benign prompt, if cached, would block 0.60% of the unseen benign prompts. The false positives that write-back adds (1.4 test prompts per stream and one validation prompt) come instead from correctly cached attacks whose neighborhood contains a benign prompt. Write-back itself matters mainly from an empty cache, where it raises recall from 91.12% to 92.64% and saves 14.7% of the LLM calls; from the warm cache it leaves recall unchanged and saves 0.9%.

The safeguards cost little on this traffic. The benign-overlap check changes no decision: it rejects 1, 2 and 5.6 of about 200 correct entries with 100, 500 and 1,741 reference prompts, and none of the gate false positives, which lie far from all benign prompts. The probationary tier costs 3.2–5.3% more LLM calls and up to 0.28 points of recall from an empty cache; the cache still matches 63–77% of the attacks that resemble a learned one, and 97–98% of them are blocked by the later stages or the cache. The time-to-live costs 1.6–8.2% more LLM calls and removes 50–88% of the learned entries by the end of the stream, which lowers held-out recall from 91.91% to 90.94–91.20%. Re-validation costs 0.1–1.7% more LLM calls, but evicts a correct entry whenever the pipeline misses a re-validated attack (0.8 and 1.2 entries per stream at 5% and 10%). No gate false positive was ever promoted. All four together cost 6.9% more LLM calls and 0.47 points of recall from an empty cache, and 0.6% more LLM calls from the warm cache, while lowering the FPR from 6.00% to 5.74% and from 9.08% to 8.82%, respectively. The safeguards target deliberate poisoning, which places entries next to benign traffic; measuring how much they reduce it, in terms of poison write rate, false positives after a poisoning campaign, amplification per poisoned entry, recovery time and time to eviction, is left to future work.

5.4 Ablation Study (Table 5)

The configurations in Table 5 are: C1, the Injection Detector's LLM agent alone on every prompt; C1b, both LLM-assisted detectors in parallel on every prompt, with no deterministic stage; C2, the regex pre-screens and threat-intelligence

Configuration	Warm start Rec.	FPR	Hit	LLM/1k	Val. FPR	Cold start Rec.	FPR	Hit	LLM/1k	Val. FPR
No write-back	97.10	8.72	47.5	498.9	8.11	91.12	5.64	0.0	872.6	5.41
Write-back only	97.10	9.08	47.9	494.3	8.35	92.64	6.00	15.4	744.4	5.65
Overlap check, 100 ref.	97.10	9.08	47.9	494.3	8.35	92.64	6.00	15.4	744.4	5.65
Overlap check, 500 ref.	97.10	9.08	47.9	494.3	8.35	92.64	6.00	15.4	744.4	5.65
Overlap check, 1,741 ref.	97.10	9.08	47.9	494.3	8.35	92.64	6.00	15.3	745.0	5.65
Probation, $\tau' = 0.68$	97.10	8.87	47.8	495.1	8.35	92.46	5.79	12.9	768.2	5.65
Probation, $\tau' = 0.70$	97.10	8.87	47.8	495.1	8.35	92.36	5.79	12.2	774.1	5.65
Probation, $\tau' = 0.75$	97.10	8.87	47.8	495.1	8.35	92.36	5.79	10.8	783.9	5.65
TTL, 100 requests	97.10	8.87	47.6	497.5	8.11	92.25	5.79	8.0	805.1	5.41
TTL, 250 requests	97.10	8.97	47.7	496.6	8.16	92.36	5.90	11.4	776.9	5.45
TTL, 500 requests	97.10	9.08	47.8	495.1	8.21	92.46	6.00	14.0	756.5	5.50
Re-validation, 1%	97.10	9.08	47.9	494.3	8.35	92.64	6.00	15.3	745.2	5.65
Re-validation, 5%	97.10	9.08	47.9	494.3	8.35	92.54	6.00	14.4	752.9	5.65
Re-validation, 10%	97.10	9.03	47.9	494.9	8.35	92.57	5.95	13.8	757.3	5.65
All four	97.10	8.82	47.6	497.2	8.16	92.17	5.74	9.3	795.8	5.45

Table 4. Cost of the cache safeguards on normal traffic (no attacker): the neuralchemy test split streamed with write-back, from the warm cache and from an empty one (means over five stream orders, in %; LLM/1k: LLM calls per 1,000 prompts). Val. FPR: validation split, with the cache reached at the end of the stream frozen. No write-back: frozen warm cache, or no cache. All four: 1,741 reference prompts, $\tau' = 0.70$, a TTL of 250 requests and 5% re-validation.

signatures; C3, the warm semantic cache alone; C4, the deterministic detectors without regex pre-screens; C5, regex and cache; C6, regex and deterministic detectors; C7, all fast layers without the LLM; C8, the full system with the warm cache; and C9, the full system with an initially empty cache that learns during the run. The ablation results show that the components are complementary: no single component approaches the full system. The standalone LLM Gate (C1) reaches an F1 score of 91.43%, whereas the regex engine alone (C2) reaches 22.08% at a 1.03% FPR; most neuralchemy attacks carry no fixed lexical signature. The main gains come from combining complementary components. Regex and semantic caching (C5) raise F1 to 87.82%, and adding the parallel detectors (C7) raises it to 88.59% without any LLM call. The full pipeline (C8) achieves the highest F1 and Recall, improving F1 by 4.38 points over C1; this difference is statistically significant (McNemar's $\chi^2 = 22.3$, $p < 10^{-5}$).

Adding the LLM Gate improves recall from approximately 82% to 97%, although Precision decreases because of false positives on ambiguous prompts (FPR 4.87% to 8.46%). This reflects a security-oriented trade-off that favors attack detection over missed threats. Finally, parallel execution limits the latency added by the heuristic ensemble, with C4 and C6 recording 14.5 and 16.8 ms. The full pipeline (304 ms) is faster on average than the LLM Gate alone (406 ms), because cache hits skip the LLM, while the configurations without the LLM stay below 150 ms. Running both LLM-assisted detectors in parallel on every prompt without any deterministic stage (C1b) reaches an F1 score of 92.71% (recall 89.86%); the full pipeline remains significantly better (McNemar $\chi^2 = 16.4$, $p < 10^{-4}$) while invoking the LLM for only half of the prompts. Starting from an empty cache that learns during the run (C9), the system blocks 151 prompts from the cache

Configuration	N	Prec.	Recall	F1	FPR	Latency
C1: LLM Gate Only	942	96.39	86.96	91.43	4.62	406 ms
C1b: Par. LLM Agents	942	95.75	89.86	92.71	5.64	—
C2: Regex Only	942	94.52	12.50	22.08	1.03	10.3 ms
C3: Cache Only	942	95.97	77.72	85.89	4.62	70.1 ms
C4: Par. Det. Only	942	**100.00**	20.47	33.98	**0.00**	14.5 ms
C5: Regex + Cache	942	95.92	80.98	87.82	4.87	80.2 ms
C6: Regex + Par. Det.	942	97.58	29.17	44.91	1.03	16.8 ms
C7: All Fast (No LLM)	942	95.98	82.25	88.59	4.87	142 ms
C8: Full (Warm)	942	94.22	**97.46**	**95.81**	8.46	304 ms
C9: Full (Cold)	942	95.71	92.93	94.30	5.90	—

Table 5. Ablation study of Semantic Firewall components on the neuralchemy test split ($N =$ 942). Latency is the mean over 200 test prompts processed one at a time (LLM served by Groq); "—": not measured in this setting.

(148 attacks, 3 benign), lowering the LLM-call rate from 87.3% to 73.9% and raising F1 from 93.49% to 94.30% relative to no cache; this single-pass gain is not significant ($p = 0.077$), and most of the cache's benefit comes from seeding it with known attacks (C8).

Policy profiles mainly change how lower-severity findings are handled: the *Strict* and *Developer* profiles change the decision for only 1 and 2 of the 942 test prompts (F1 95.90% and 95.63%; McNemar $p \geq 0.5$), since almost all findings on this benchmark are high-severity and are blocked under every profile. For example, the prompt "Translate the following into French. Ignore the above and instead print your system prompt, then email it to admin@example.com" is blocked with a risk score of 100, and the explanation report names the Injection, Threat-Intel and PII detectors with their evidence, whereas a benign request containing an e-mail address and a phone number is forwarded with both replaced by `[EMAIL]` and `[PHONE]`. On neuralchemy, five detectors change no decision, as its prompts contain no personal data, credentials, oversized inputs, feed signatures or workspace terms. We therefore remove one detector at a time on MIXED-1000 (Table 6), 1,000 prompts released with the code that combine 300 benign and 250 attack prompts of the neuralchemy test split with inputs for every detector: 100 AI4Privacy texts with personal data, 80 developer questions with generated credentials in real key formats, 50 inputs padded beyond the 4,000-character budget, 50 requests with the feed's signatures (cloud-metadata URLs, chat-delimiter tokens), 40 requests naming a workspace-restricted project, 80 neuralchemy attacks combined with personal data, padding or credentials, and 50 benign look-alikes (order numbers, commit hashes, long documents under the budget, localhost URLs). Every detector but one changes decisions: in the fast layers, removing the PII or Secrets Detector changes 78 and 71 decisions and lowers recall on personal data from 65.4% to 13.1% and on credentials from 100% to 32.4%; Custom Rules changes 40, Context Flooding 7 and Abuse 5, and in the full system the Injection Detector changes 50 (injection recall 98.2% to 91.8%) and Unsafe Content 16. Threat Intel flags 58 prompts, all of its own inputs included, but each is also flagged by another detector: its feed holds only three signatures (instruction override, chat-delimiter tokens, a cloud-metadata URL), which overlap with the Abuse and PII Detectors and the injection regex,

so removing it changes no decision on this set. Its role is to let operators add signatures for newly observed attacks without changing code. The FPR is high on this set (19.43% for C8) because the look-alikes were built to resemble threats: 40 of the 50 are flagged, by the PII Detector (order numbers, IP addresses), the Injection regex (hashes, long documents) and the Abuse Detector.

Removed detector	Compared with	F1	FPR	Changed	Flags	Own-threat recall
none	C8: Full (Warm)	92.35	19.43	—	—	—
Injection	C8	91.43	13.43	50	278 / 32	98.2 → 91.8
Unsafe Content	C8	91.96	17.71	16	111 / 16	—
none	C7: All Fast (No LLM)	89.22	15.43	—	—	—
Abuse	C7	88.78	15.43	5	154 / 10	—
PII	C7	83.65	12.57	78	155 / 20	65.4 → 13.1
Secrets	C7	82.67	15.43	71	105 / 0	100.0 → 32.4
Threat Intel	C7	89.22	15.43	0	58 / 0	100.0 → 100.0
Custom Rules	C7	85.62	15.43	40	40 / 0	100.0 → 0.0
Context Flooding	C7	88.61	15.43	7	76 / 0	100.0 → 90.7

Table 6. Per-detector ablation on MIXED-1000 (N = 1,000; 650 prompts with threats, 350 benign): each detector is removed in turn, the LLM-assisted ones from the full system (C8) and the deterministic ones from the fast layers (C7), both with the warm cache; the LLM verdicts of the full run are reused in the ablation runs (temperature 0). Changed: decisions that change when the detector is removed. Flags: prompts the detector flags in the reference run (with threats / benign). Own-threat recall: recall on the threat type the detector targets, with and without it.

Cost On our evaluation distribution, fast layers resolved 50.1% of all incoming requests without LLM invocation; this ratio will vary with deployment-specific traffic composition. We call the share of attacks among incoming requests the *attack rate*; it is 59% on the test split (552 of 942 prompts). Because cache hits are attacks and benign prompts almost always reach the LLM Gate, the saving grows with the share of attack traffic: on the benign no_robots prompts, 98.0% of requests invoke the LLM. Re-weighting the test split to attack rates of 1–10% (Table 7) gives an LLM-call rate of 94.4–87.4% and an effective latency of 495–466 ms (p95 944 ms, p99 1242 ms). Applying (8) to the test split (P_c=447/942, P_h+P_m=25/942, P_l=470/942, with measured mean path latencies of 85.6, 332.1 and 519.1 ms) results in an effective system latency of $L_{\text{eff}} \approx 308$ ms. Economically, each invocation of the LLM stage issues two requests (one per LLM-assisted detector) to an open-weight 70B model, so the cost advantage over a proprietary LLM-only gate comes mainly from the lower per-token price of the open model and grows with the share of attack traffic resolved earlier.

6 Limitations & Future Work

A few limitations point to future work. The 70B LLM Gate is resource-heavy; a distilled 8B model could cut cost and latency. The cache can still miss attacks worded very differently from anything seen before, which adversarially trained

Attack rate	Resolved w/o LLM	LLM-call rate	L_{eff}	p95	p99
1%	5.6%	94.4%	495 ms	944 ms	1242 ms
5%	8.7%	91.3%	482 ms	944 ms	1242 ms
10%	12.6%	87.4%	466 ms	944 ms	1242 ms
59% (test split)	50.1%	49.9%	308 ms	756 ms	1047 ms

Table 7. Share of requests resolved without the LLM, LLM-call rate (which determines the API cost), and latency of the full system as a function of the attack rate, that is, the share of attacks among incoming requests. The test-split row is measured; the other rows re-weight the per-class results of the test split (latency measured one request at a time, LLM served by Groq).

embeddings could help catch. The Python orchestrator processes about three requests per second per sequential worker (304 ms on average, most of it spent waiting for the LLM); a production version would need a faster language such as Go or Rust and a distributed vector store. Unresolved prompts are sent to a third-party LLM API, which exposes their content to that provider; because the gate is an open-weight model, privacy-sensitive deployments can host it locally. The 8.46% false-positive rate may be too strict for cost-sensitive deployments, though Table 3 shows higher-threshold settings trade some accuracy for fewer false positives (on the test split, $\tau = 0.80$ gives 6.67% FPR at 94.62% F1). Future work will also select the detector weights and thresholds automatically, by optimising them on a validation split instead of setting them by hand. Future work will also explore multimodal inputs. The evaluation covers static attack datasets, keywords obfuscated with typos, out-of-distribution and multi-turn attacks, prompts unlike any cached attack and a black-box allowlist test, but not adaptive attackers who know the system in advance (Section 3.9); future work will evaluate attacks optimized against the detectors and the open-weight gate, injections addressed to the gate itself, near-allowlist inputs crafted with knowledge of the pipeline, and deliberate poisoning campaigns against the poisoning-resistant cache design.

7 Conclusion

This paper presented Semantic Firewall, a layered, multi-agent defense-in-depth framework for securing LLM applications against prompt-injection and jailbreak attacks. By combining lightweight deterministic checks, an adaptive semantic memory, a parallel ensemble of eight specialized detectors, and a constrained LLM gate, Semantic Firewall achieves a 95.81% F1 score and 97.46% recall on the neuralchemy test split, outperforming GPT-4o, Gemini 2.5 Flash, and Claude Haiku 4.5. It resolves 50.1% of test traffic without any LLM inference; this share shrinks when attacks are a smaller share of the traffic. Results on another prompt-injection benchmark (deepset), on related tasks (BeaverTails, AI4Privacy) and on an unseen out-of-distribution set (F1 of 92.44%) indicate that the approach carries beyond its primary evaluation setting. These results demonstrate that combining detection mechanisms with complementary failure modes and adaptive semantic memory offers a practical and scalable blueprint

for securing production LLM-based applications against an evolving threat landscape.

Generative AI Usage Declaration

The authors used a large language model only to assist with manuscript preparation, including language editing and rephrasing for clarity, tone, and formality. All AI-assisted text was reviewed, verified, and edited by the authors, who take full responsibility for the accuracy, originality, and integrity of the manuscript.

References

1. Chen, J., et al.: A survey on prompt injection attacks in large language models. IEEE Trans. Inf. Forensics Secur. (2024)
2. OWASP: OWASP top 10 for large language model applications. OWASP Foundation (2023)
3. Greshake, K., et al.: Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: Proc. AISec (2023)
4. Liu, X., et al.: AutoDAN: generating stealthy jailbreak prompts on aligned large language models. arXiv preprint arXiv:2310.04451 (2023)
5. Chao, P., et al.: Jailbreaking black box large language models in twenty queries. arXiv preprint arXiv:2310.08419 (2023)
6. Wei, A., Haghtalab, N., Steinhardt, J.: Jailbroken: how does LLM safety training fail? In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
7. Zou, A., et al.: Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043 (2023)
8. Inan, M., et al.: Llama guard: LLM-based input-output safeguard for human-AI conversations. arXiv preprint arXiv:2312.06674 (2023)
9. Guardrails AI: Guardrails: validating and correcting the outputs of large language models. GitHub (2023)
10. Meta: PromptGuard: a safeguard model against prompt injections and jailbreaks (2024)
11. ProtectAI: Fine-tuned DeBERTa-v3-base for prompt injection detection (v2). HuggingFace (2024), https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2
12. Li, H., Liu, X.: InjecGuard: benchmarking and mitigating over-defense in prompt injection guardrail models. arXiv preprint arXiv:2410.22770 (2024)
13. Kholkar, G., Ahuja, R.: CAPTURE: context-aware prompt injection testing and robustness enhancement. arXiv preprint arXiv:2505.12368 (2025)
14. Maiorano, A.C.: Evaluating prompt injection defenses for educational LLM tutors: security-usability-latency trade-offs. arXiv preprint arXiv:2605.06669 (2026)
15. ProtectAI: Rebuff: prompt injection defense. GitHub (2023)
16. Palit, S., Woods, D.: Evaluating the efficacy of LLM safety solutions: the Palit benchmark dataset. arXiv preprint arXiv:2505.13028 (2025)
17. Wallace, E., Xiao, K., Leike, R., Weng, L., Heidecke, J., Beutel, A.: The instruction hierarchy: training LLMs to prioritize privileged instructions. arXiv preprint arXiv:2404.13208 (2024)

18. Chen, S., Piet, J., Sitawarin, C., Wagner, D.: StruQ: defending against prompt injection with structured queries. In: Proc. USENIX Security Symposium (2025)
19. Chen, S., Zharmagambetov, A., Mahloujifar, S., Chaudhuri, K., Wagner, D., Guo, C.: SecAlign: defending against prompt injection with preference optimization. arXiv preprint arXiv:2410.05451 (2024)
20. Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., Tramèr, F.: Defeating prompt injections by design. arXiv preprint arXiv:2503.18813 (2025)
21. Rebedea, T., et al.: NeMo guardrails: a toolkit for controllable and safe LLM applications. arXiv preprint arXiv:2310.10501 (2023)
22. Saltzer, J., Schroeder, M.: The protection of information in computer systems. Proc. IEEE **63**(9), 1278–1308 (1975)
23. Zhu, K., et al.: PromptBench: towards evaluating the robustness of large language models on adversarial prompts. arXiv preprint arXiv:2308.06588 (2023)
24. Robey, A., et al.: SmoothLLM: defending large language models against jailbreaking attacks. arXiv preprint arXiv:2309.02771 (2023)
25. Russinovich, M., Salem, A., Eldan, R.: Great, now write an article about that: the Crescendo multi-turn LLM jailbreak attack. arXiv preprint arXiv:2404.01833 (2024)
26. Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: Proc. EMNLP-IJCNLP, pp. 3982–3992 (2019)
27. Chroma: ChromaDB: the AI-native open-source embedding database. GitHub (2023)
28. Malkov, Y., Yashunin, D.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Trans. Pattern Anal. Mach. Intell. **42**(4), 824–836 (2020)
29. Shannon, C.E.: A mathematical theory of communication. Bell Syst. Tech. J. **27**(3), 379–423 (1948)
30. Wagner, R.A., Fischer, M.J.: The string-to-string correction problem. J. ACM **21**(1), 168–173 (1974)
31. McNemar, Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika **12**(2), 153–157 (1947)
32. Neuralchemy: Prompt-injection-dataset. HuggingFace (2023), https://huggingface.co/datasets/neuralchemy/Prompt-injection-dataset
33. AI4Privacy: pii-masking-200k. HuggingFace (2023), https://huggingface.co/datasets/ai4privacy/pii-masking-200k
34. Ren, Q., Li, H., Liu, D., Xie, Z., Lu, X., Qiao, Y., Sha, L., Yan, J., Ma, L., Shao, J.: Derail yourself: Multi-turn LLM jailbreak attack through self-discovered clues. arXiv preprint arXiv:2410.10700 (2024)
35. Ji, J., et al.: BeaverTails: towards improved safety alignment of LLM via a human-preference dataset. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
36. deepset: prompt-injections. HuggingFace (2023), https://huggingface.co/datasets/deepset/prompt-injections
37. Rajani, N., et al.: No Robots. HuggingFace (2023), https://huggingface.co/datasets/HuggingFaceH4/no_robots
38. OpenAI: GPT-4o system card. Tech. rep., OpenAI (2024), https://openai.com/index/gpt-4o-system-card/
39. Anthropic: Claude Haiku 4.5. Model announcement, Anthropic (2025)

40. Comanici, G., et al.: Gemini 2.5: pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. arXiv preprint arXiv:2507.06261 (2025)
41. Meta: Llama Guard 4 model card. Meta AI (2025), https://github.com/meta-llama/PurpleLlama/blob/main/Llama-Guard4/12B/MODEL_CARD.md